

SOLVETAXADMIN · INTERNAL TECHNICAL REFERENCE

WhatsApp Integration

Developer Handbook

This handbook documents the node-based WhatsApp workflow builder and execution engine built into SolvetaxAdmin on top of the free, self-hosted Evolution API (Baileys channel). It covers architecture, the guardrail chain, the canvas and engine internals, how to run locally, and how to extend the system.

Version 1.0 · PR #9 → develop (merged-pending)

Date 24 July 2026

Platform Evolution API v2.3.7 · Baileys channel · Free / self-hosted · Zero per-message cost

Table of Contents

1	Summary + Architecture	3
2	How Notifications Reach a Customer	4
3	Guardrail Chain	6
4	Canvas + Engine	7
5	Local Setup	11
6	Extending the System	12
7	Roadmap + API Reference	12
G	Glossary	14

1. Summary + Architecture

What was built

- React 19 canvas** at `/whatsapp-flows` — flow list, full-screen editor (`@xyflow/react v12`), node palette, config drawer, validate modal with jump-to-node.
- FastAPI backend** — full CRUD + publish + validate + simulate under `/api/v1/whatsapp/` (raw SQL / asyncpg; no ORM).
- Three-step execution engine** (steps 14a / 14b / 14c) appended to the existing asyncio scheduler — no new infrastructure, no queue daemon, no extra Azure container.
- Guardrail chain** in `send_service.py`: consent → quiet hours → daily cap — enforced for every send regardless of trigger source.
- Five new DB tables** across three migrations (V003–V005): `wa_consent`, `wa_instance_config`, `wa_flows`, `wa_flow_runs`, `wa_outbox`.
- 94 backend tests** (first `pytest` infra in this repo) + CI job with live Postgres + Redis services.

Status: PR #9 → develop (merged-pending). The `ScheduledDate`-triggered flow shape is fully implemented and E2E verified. `InboundKeyword` trigger and `Wait(reply)` handler are scaffolded on canvas; backend handlers deferred to Slice 3. Campaign broadcasts and two-way chat are Phase 2.

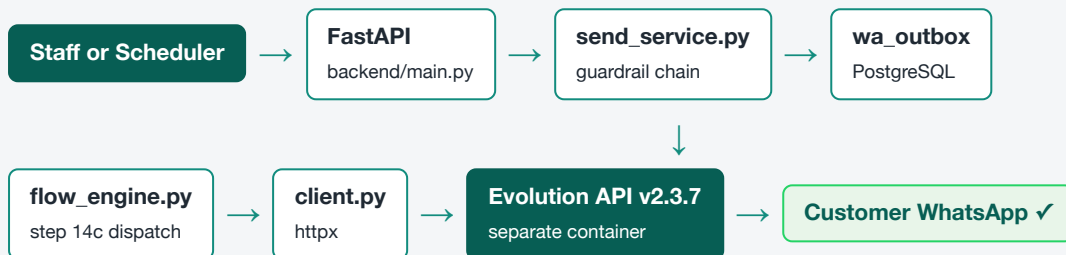
Stack

Layer	Technology	Notes
Backend API	FastAPI + asyncpg	No ORM. Parameterised SQL strings. Schema: <code>solvetax</code> .
Database	PostgreSQL	Azure Database for PostgreSQL Flexible Server.
Cache / lock	Redis	Scheduler leader election; daily send counter.
Frontend	React 19 + Vite SPA	Served from same Azure App Service via static build.
Canvas	<code>@xyflow/react v12</code> (MIT)	React 19-compatible since Jan 2025.
Deployment	Single Azure App Service B1	Backend + frontend co-located. 1.75 GB RAM.
WhatsApp gateway	Evolution API v2.3.7	Separate self-hosted container (Baileys channel). Zero per-message cost.

Module map

```
backend/whatsapp/  
  router.py / flows_router.py  # FastAPI APIRouter - all /api/v1/whatsapp/* endpoints  
  client.py                   # httpx wrapper for Evolution API HTTP calls  
  send_service.py             # Guardrail chain + outbound send logic  
  sinks.py                    # EvolutionSink / DryRunSink (MessageSink protocol)  
  flow_validation.py          # 6-check validator + cycle detection  
  flow_engine.py              # Steps 14a/14b/14c: enrollment, tick, outbox dispatch  
  variable_resolver.py        # {{token}} whitelist resolver  
  tests/                      # 94 pytest tests (first test infra in repo)  
  
frontend/src/components/whatsapp/  
  WhatsAppConfig.jsx / WhatsAppFlowList.jsx / WhatsAppFlowEditor.jsx
```

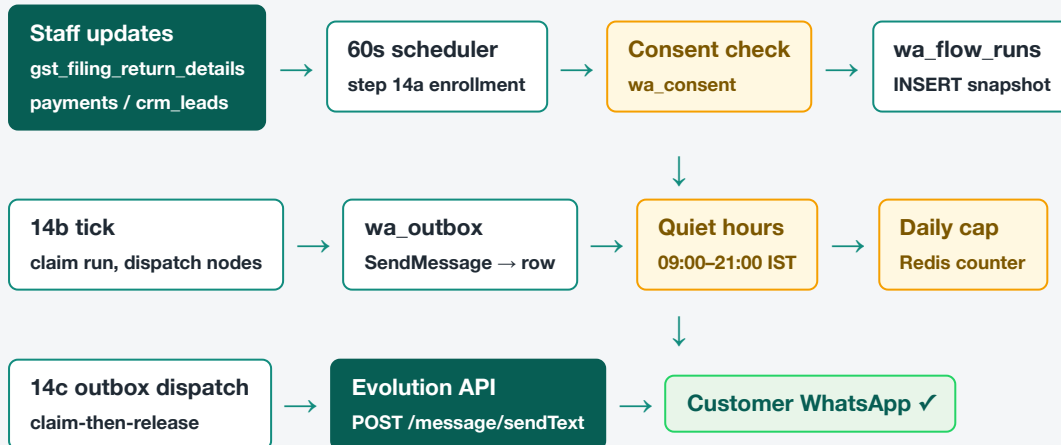
Request / data flow



2. How Notifications Reach a Customer

Staff change nothing in their routine. They update the CRM tables they already maintain — GST due dates, payment follow-ups, lead timestamps — and the engine handles enrollment and delivery. A flow is authored *once* on the canvas.

DATA FLOW — CRM update → WhatsApp delivery



GSTR-3B reminder journey — end to end

The proving journey. A single `wa_flows` row with a `ScheduledDate` trigger on `gstr3b_due_date` with `days_before=7` drives the full sequence.

When	Trigger / Condition	Engine action	DB effect
T-7 days	<code>gstr3b_due_date</code> 7 d out; consent present	14a enrolls; CRM snapshot → <code>context</code>	<code>wa_flow_runs</code> INSERT
T-7 (same tick)	<code>SendMessage</code> — 7-day reminder	14b writes outbox row, advances node	<code>wa_outbox</code> INSERT
T-7 (next dispatch)	Outbox <code>pending</code> ; within quiet hours; under cap	14c calls Evolution API	<code>wa_outbox.status</code> → <code>sent</code>
T-5 days	<code>Wait(delay, 2d)</code> wakes → Condition: <code>filing_status != 'FILED'</code>	Re-queries <code>gst_filing_return_details</code> anchored to <code>__source_row_id</code>	LIVE SQL re-read
— filed	<code>filing_status == 'FILED'</code>	Follows <code>false_output</code> → <code>EndFlow</code> ; silent exit	<code>wa_flow_runs.status</code> → <code>completed</code>
— unfiled	<code>filing_status != 'FILED'</code>	Follows <code>true_output</code> → <code>SendMessage</code> + <code>AssignTask</code>	<code>wa_outbox</code> INSERT; <code>employee_tasks</code> INSERT
Follow-up	Another <code>Wait(2d)</code> + Condition	Same pattern; second follow-up or RM escalation	Runs until <code>EndFlow</code>

⚠ **Condition node re-reads live data.** All other nodes read the CRM snapshot captured at enrollment. Condition nodes are the deliberate exception — they re-query the source row anchored to `context.__source_row_id` at evaluation time. This is what makes "wait then check if still unfiled" work correctly.

The 4 gates every message must pass

#	Gate	Where enforced	Blocks on
1	Mobile number present	Step 14a enrollment SQL	No <code>customers.mobile</code> → skipped with DEBUG log
2	Active consent row	Step 14a + re-checked in <code>send_service.send()</code>	No row or <code>revoked_at IS NOT NULL</code> → never enrolled; mid-run revoke → outbox <code>cancelled</code>
3	Published & active flow	Step 14a: <code>WHERE status='published' AND is_active=true</code>	Draft/archived → not queried; <code>is_active=false</code> → paused
4	Evolution API instance connected	Step 14c dispatcher	0 or >1 active instances → dispatch skipped this tick with ERROR log

NAME	TRIGGER	STATUS	ACTIVE	VERSION	UPDATED
C2B1B-1B Reminder	Scheduled Date	PUBLISHED	☒	1	24 Jul 2020

Figure 4. Flow list at `/whatsapp-flows`. Published flows show a green badge. The `is_active` toggle pauses enrollment without archiving the flow. Only published + active flows are queried by step 14a.

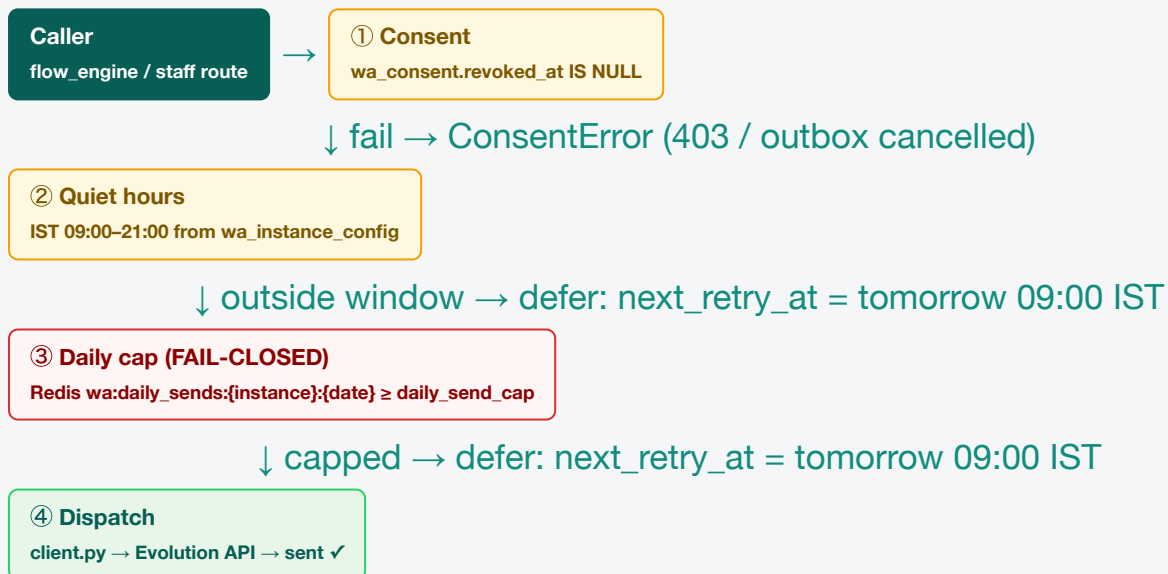
NAME	TRIGGER	STATUS	ACTIVE	VERSION	UPDATED
C2B1B-1B E2E	Scheduled Date	PUBLISHED	☒	1	24 Jul 2020

Figure 5. Flow list during E2E testing against a mock Evolution gateway. Confirms live scheduler enrollment ran, token-resolved messages dispatched over real HTTP, and the filed-during-wait branch exited silently.

3. Guardrail Chain

Every outbound message passes through `send_service.send()` regardless of trigger source — flow engine, direct staff send, or future integrations. No flow can bypass guardrails.

`send_service.send()` — decision chain



Gate — FAIL-CLOSED Redis convention exception. The rest of the codebase uses fail-open Redis (`redis_cache.py`): if Redis is unavailable, operations proceed. The daily-cap check ③ is the one deliberate exception — if Redis is unavailable the cap check **blocks all sends** for that tick. An uncounted send is a ban-risk send; this trade-off is intentional.

All guard thresholds are stored in `wa_instance_config` (single DB row) — no code redeploy needed to adjust warm-up caps:

```
UPDATE solvetax.wa_instance_config SET daily_send_cap = 100 WHERE instance_name = 'primary';
```

HTTP error mapping

Guardrail	HTTP code (manual send)	Outbox effect	Resolution
No consent / revoked	403 Forbidden	Row → cancelled	Capture consent first
Missing required field	422 Unprocessable Entity	—	Check request body
Daily cap reached	429 Too Many Requests	Deferred to next day	Wait; or raise cap in <code>wa_instance_config</code>
Quiet hours	422 (time check before outbox)	Deferred to 09:00 IST	Retry after window opens
Evolution API error	502 Bad Gateway	Retry x5 exponential backoff	Check instance connection state

Warm-up ladder: Weeks 1–2: 50/day (default cap), pilot batch 20–50 consented clients. Weeks 3–4: raise to 100/day via `UPDATE wa_instance_config SET daily_send_cap = 100`. Month 2+: raise to 200–500/day monthly if no incidents; `asyncio.sleep(2)` intra-send pacing avoids volume spikes.

⚠ **Baileys ban risk.** 68% of Indian SMB accounts using unofficial automation report at least one number ban within 12 months. Ban during GST/ITR season is operationally catastrophic. Operate at minimum effective volume. Never use the main business SIM.

4. Canvas + Engine

How a flow serializes

Flows are stored as `reactFlowInstance.toObject()` — a JSONB object with three top-level keys. No custom serialization layer exists.

```
// wa_flows.draft_data / live_data — reactFlowInstance.toObject() shape
{ "nodes": [ { "id": "node-abc",
  "type": "waNode",           // ALL nodes share this React component type
  "data": { "nodeType": "scheduledDate", // engine dispatches on data.nodeType
    "config": { "source": "gstr3b_due_date", "days_before": 7 } } },
  "edges": [ { "id": "e1", "source": "node-abc", "target": "node-def" } ],
  "viewport": { "x": 0, "y": 0, "zoom": 1 } }
```

⚠ **Critical shape detail — `type` vs `data.nodeType`**. All canvas nodes carry `type: "waNode"` (the React component). The logical type the engine dispatches on is `data.nodeType`. Config payload is `data.config`. Getting this wrong caused two separate bugs during development. Always read from `node["data"]["nodeType"]` and `node["data"]["config"]`.

`draft_data` is auto-saved every 2 s (debounced). `live_data` is set only by the publish action and is the only field the engine ever reads.

Nine node types (v1)

Node type	Cat.	Purpose	Key config
InboundKeyword	Trigger	Fires on matching inbound client message	keyword, match_mode (contains/equals/regex)
ScheduledDate	Trigger	Fires when a CRM date field falls within look-ahead window	source (field name), days_before
CrmEvent	Trigger	Fires on a CRM state change via post-commit hook	event_type, from_stage / to_stage
SendMessage	Action	Writes a wa_outbox row; dispatched by outbox worker	body with {{variable}} tokens
AssignTask	Action	Creates an employee_tasks row	assignee (RM_OF_CUSTOMER / OP_OF_CUSTOMER), title, description
UpdateCRMField	Action	Parameterised UPDATE on safe crm_leads columns	field, value
Condition	Logic	Single comparison on a context variable (live re-read)	variable, operator (eq/neq/contains/gt/lt), value
Wait	Logic	Pauses run for a delay or until a reply	type (delay/reply), delay_minutes, timeout_hours
EndFlow	Control	Marks run status='completed'	None

12-token {{variable}} whitelist

Resolved from CRM context snapshot in `variable_resolver.py`. Unrecognised tokens are left as-is. The config drawer shows a static dropdown scoped to the active trigger type.

```
customer_name · gst_number · gstr3b_due_date · gstr1_due_date
payment_amount_due · payment_due_date · rm_name · op_name
filing_status · pipeline_stage · income_tax_year · pending_documents_count
```

Server-side validation (6 checks)

Run by `/validate` (real-time) and `/publish` (gate). FlowEditor calls `/validate` debounced for red-ring feedback + "Jump to node" UX.

- Exactly one trigger node (InboundKeyword, ScheduledDate, or CrmEvent).
- All non-start nodes reachable from start (orphaned nodes → red ring).
- Every Condition node has both true_output and false_output handles connected.
- Every Wait(reply) node has both on_reply and on_timeout handles connected.
- At least one EndFlow reachable from start.
- All required config fields non-empty (per-node-type `_REQUIRED_CONFIG` map).

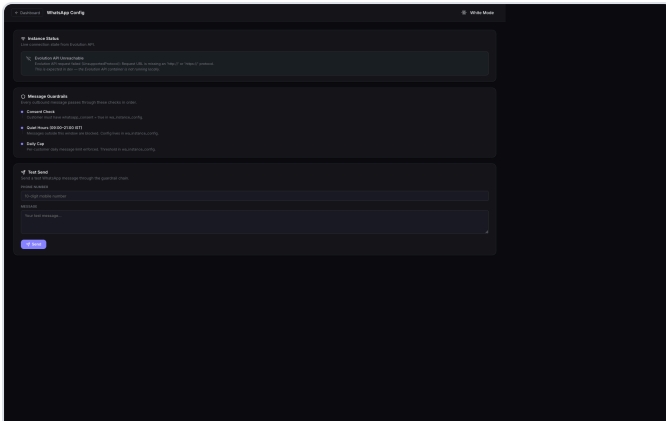


Figure 1. Config page — dark theme. Shows Evolution API instance connection status, API key, quiet-hours display, and daily cap from `wa_instance_config`.

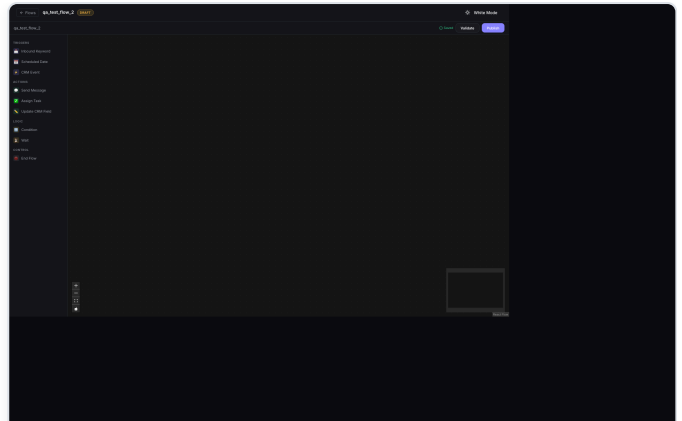


Figure 2. Flow editor — dark theme. Left: draggable node palette (Triggers / Actions / Logic / Control). Right: config drawer (opens on node click). Bottom: Validate, Publish, Simulate, `is_active` toggle.

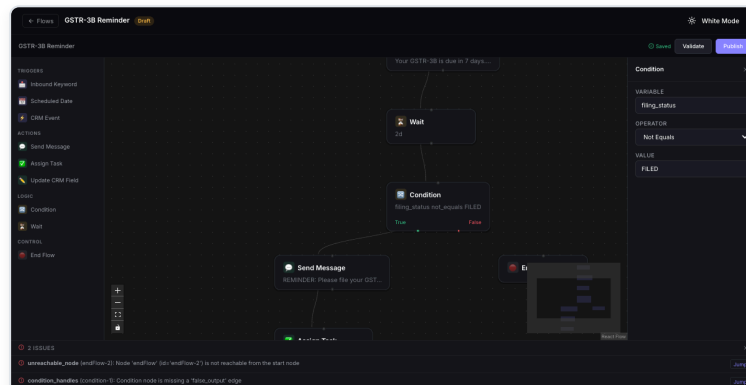


Figure 3. Validate modal with detected issues. Each row has a "Jump to node" button that calls `reactFlowInstance.setCenter()`. Same logic runs server-side at publish — the modal uses the `/validate` endpoint, not a client-side reimplementation.

Execution engine — steps 14a / 14b / 14c

Three functions in `flow_engine.py` sharing the existing asyncio event loop, asyncpg pool, and Redis client.

Step 14a — `_start_scheduled_flows(conn)` (enrollment)

- Runs every 60 s. For each published, active `ScheduledDate` flow: reads node config → builds parameterised join query → checks `wa_consent` → skips if outside quiet hours → inserts `wa_flow_runs` with CRM snapshot.
- Join topology by source:** `gstr3b/gstr1_due_date` → `gst_filing_return_details` → `gst_filings` → `customers`; `payment_followup_at` → `payments` → `customers`; `crm_followup_at` → `crm_leads.mobile` → `customers.mobile`.
- Dedup:** excludes customers with an existing `wa_flow_runs` row (any status) for this flow in the current trigger period. Partial unique index `idx_wa_flow_runs_active` is the concurrent-insert guard.
- Batch cap: `SCHEDULER_SQL_BATCH_LIMIT = 500`. Shared-mobile guard: skips (WARN log) customers with multiple matching `customers` rows.
- CrmEvent* flows are enrolled via post-commit `_maybe_start_flow()` — they do not go through step 14a.

Step 14b — `_tick_wa_flow_runs(conn)` (tick)

- Claims 10 runnable runs per tick using `FOR UPDATE SKIP LOCKED`.
- Reads `current_node_id` from `context.__flow_def` snapshot (never joins back to `wa_flows`).
- Dispatches to matching Python handler. Handlers chain synchronously until a halt point.
- Halt points:** `Wait` (sets `wake_at`, `status='waiting'`); `EndFlow` (sets `status='completed'`).
- `SendMessage` is a **pass-through** — writes outbox row, advances `current_node_id`, continues same tick.
- All state writes per run commit or roll back together. Crash releases `FOR UPDATE` lock on rollback.

- **Stale-run reaper:** runs where `status='running' AND heartbeat_at < now() - 15 min` → `status='failed'`. `status='waiting'` runs are intentionally excluded.

Step 14c — `_dispatch_outbox(conn)` (outbox dispatch)

- Short claim transaction sets `status='sending'` on up to 5 pending rows and **commits immediately** — locks held milliseconds only.
- HTTP sends run lock-free with 10 s timeout + `await asyncio.sleep(2)` intra-send pacing.

Handler	Halt?	Key behaviour
<code>scheduledDate</code>	No	Entry point — validates context populated; advances to next node.
<code>sendMessage</code>	No (pass-through)	Resolves <code>{{variables}}</code> ; writes <code>wa_outbox</code> row; advances. Evolution API NOT called here.
<code>wait</code> (delay)	Yes	Sets <code>wake_at = now() + delay_minutes</code> ; <code>status='waiting'</code> .
<code>condition</code>	No	Live re-reads source row via <code>__source_row_id</code> ; follows <code>true_output</code> or <code>false_output</code> .
<code>assignTask</code>	No	Resolves RM/OP from snapshot; calls <code>create_task_for_emp()</code> .
<code>endFlow</code>	Yes	Sets <code>status='completed'</code> . Customer can be re-enrolled next period.

Outbox event	Action
Success (201)	<code>status='sent'</code> ; record <code>evolution_msg_id</code> ; increment Redis daily counter.
Failure / timeout	Increment <code>retry_count</code> ; exponential <code>next_retry_at</code> . After 5 retries: <code>status='failed'</code> .
Stuck in <code>'sending'</code> >5 min	Re-queue via <code>next_retry_at</code> . Idempotency key prevents double-send.
Daily cap reached	<code>next_retry_at = tomorrow 09:00 IST</code> ; stop dispatching this tick.
0 or >1 active instances	Skip dispatch entirely; log at ERROR level.

Idempotency key format: flow-engine sends: `'{flow_run_id}:{node_id}'`; direct sends: `'{category}:{entity_id}:{IST-date}'`. `wa_outbox.idempotency_key` is UNIQUE; `INSERT ... ON CONFLICT DO NOTHING` is the dedup guard.

Simulation: `POST /flows/{id}/simulate` runs the engine to completion inside a transaction that is *always rolled back* — no state persists, no Evolution API calls made (uses `DryRunSink`). Returns `{ "trace": [...], "would_send": [...] }`.

Flow versioning: At run creation, `context.__flow_def = flow.live_data` copies the full graph. Publishing overwrites `live_data`; in-flight runs carry their own snapshot and are unaffected. Snapshot: ~3–8 KB per run row at 9–15 nodes.

Data model — five `wa_*` tables

Table	Key columns	Purpose
<code>wa_consent</code>	<code>phone</code> TEXT PK, <code>customer_id</code> BIGINT, <code>source</code> ENUM, <code>consented_at</code> , <code>revoked_at</code>	Gate 2. Must have a row with <code>revoked_at</code> IS NULL before any send. Source enum: <code>STAFF_RECORDED</code> / <code>OPT_IN_LINK</code> / <code>ONBOARDING_FORM</code> .
<code>wa_instance_config</code>	<code>instance_name</code> TEXT PK, <code>daily_send_cap</code> INT DEFAULT 50, <code>quiet_hours_start</code> INT DEFAULT 9, <code>quiet_hours_end</code> INT DEFAULT 21, <code>is_active</code> BOOLEAN	Runtime-tunable guardrail thresholds. One row per Evolution API instance. No redeploy needed to adjust warm-up caps.
<code>wa_flows</code>	<code>id</code> UUID PK, <code>name</code> TEXT, <code>trigger_type</code> TEXT, <code>status</code> TEXT (draft/published/archived), <code>is_active</code> BOOLEAN, <code>draft_data</code> JSONB, <code>live_data</code> JSONB, <code>version</code> INT	One row per flow definition. Engine reads only <code>live_data</code> . Publishing copies draft → live and increments <code>version</code> .
<code>wa_flow_runs</code>	<code>id</code> UUID PK, <code>flow_id</code> UUID, <code>customer_id</code> BIGINT, <code>phone</code> TEXT, <code>status</code> TEXT (running/waiting/completed/failed/cancelled), <code>current_node_id</code> TEXT, <code>context</code> JSONB, <code>wake_at</code> TIMESTAMPTZ, <code>heartbeat_at</code> TIMESTAMPTZ	One row per customer per run. <code>context</code> holds CRM snapshot + <code>__flow_def</code> (full graph) + <code>__source_row_id</code> . Partial unique index (<code>flow_id</code> , <code>customer_id</code>) WHERE <code>status</code> IN ('running', 'waiting') prevents concurrent re-enrollment.
<code>wa_outbox</code>	<code>id</code> UUID PK, <code>flow_run_id</code> UUID (NULL for direct sends), <code>phone</code> TEXT, <code>body</code> TEXT, <code>status</code> TEXT (pending/sending/sent/failed/cancelled), <code>retry_count</code> INT, <code>next_retry_at</code> TIMESTAMPTZ, <code>idempotency_key</code> TEXT UNIQUE, <code>purge_after</code> TIMESTAMPTZ (30 d)	Decouples Evolution API HTTP calls from scheduler tick. Single table for all outbound sends. DPDP data minimisation: <code>body</code> purged after 30 days.

Migrations

Migration file	Tables	Notes
<code>V003__wa_consent.sql</code>	<code>wa_consent</code> , <code>wa_instance_config</code>	Seed row: <code>INSERT INTO wa_instance_config (instance_name, daily_send_cap) VALUES ('primary', 50);</code>
<code>V004__wa_flows.sql</code>	<code>wa_flows</code>	GIN index on <code>draft_data</code> + <code>live_data</code> for future JSONB queries.
<code>V005__wa_flow_engine.sql</code>	<code>wa_flow_runs</code> , <code>wa_outbox</code>	Partial unique index for enrollment dedup; dispatch index on <code>wa_outbox</code> .

i Migration files were renumbered from V002/003/004 to V003/004/005 after a collision with a concurrent migration on the `develop` branch. Always check `db/migrate/migration-order.yaml` for the current highest V number before writing a new migration.

5. Local Setup

Prerequisites: Postgres 15+, Redis 7+, Python 3.11+, Node 20+, Docker.

```
# Database setup
createdb solvetaxadmin_local
export DB_HOST=localhost DB_PORT=5432 DB_NAME=solvetaxadmin_local
export DB_USER=<pg-user> DB_PASSWORD=<pg-pass> DB_SCHEMA=solvetax
export DB_SSL=disable # local only - do not set in production
python db/migrate/run_migrations.py

# Backend
export REDIS_SSL=false EVOLUTION_API_URL=http://localhost:8080
export EVOLUTION_API_KEY=<key> JWT_SECRET=<any-local-secret>
uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload

# Frontend (Vite proxy hardcoded to 127.0.0.1:8000 - backend must bind this exact address)
cd frontend && bunx vite # or: npx vite
```

Evolution API sandbox

```
cd deploy/evolution && docker compose up -d
open http://localhost:8080/manager # Manager UI
```

1. Create a new instance named **primary** in the Manager UI.
2. Scan the QR code with a **spare SIM** — never the main business number.
3. Verify `GET /instance/connectionState/primary` returns `"open"`.
4. Run a test send from the Config page (Figure 1) to confirm end-to-end delivery.

 **Gate — Never use the main business SIM for the sandbox instance.** The QR scan links the phone number to the Baileys session. A dedicated spare SIM with no prior WhatsApp history is mandatory.

```
# Run the test suite (94 tests - asyncpg transaction-rollback fixture, no test data persists)
pytest backend/whatsapp/tests/ -v
```

 **Gate — Consent capture is a hard pre-production gate.** No outbound send to a client phone may happen without a `wa_consent` row with `revoked_at IS NULL`. The consent-capture UI has not been built yet — this is the owner's responsibility before any production send. Staff can record consent manually for a pilot batch; `source` enum documents the capture method: `STAFF_RECORDED / OPT_IN_LINK / ONBOARDING_FORM`.

6. Extending the System

- **New node type:** (1) Add handler to `_dispatch_node()` in `flow_engine.py` — receives `(run, node_config, conn, sink)`, returns next `node_id` or `None`. (2) Add to `_REQUIRED_CONFIG` in `flow_validation.py`. (3) Add React component + handles in `WhatsAppFlowEditor.jsx`. (4) Add config drawer fields. Add a CI fixture in `tests/fixtures/`.
- **New `{{variable}}` token:** (1) Add to whitelist in `variable_resolver.py`. (2) Populate value in step 14a enrollment SQL — missing snapshot value means empty at send time. (3) Add to config drawer dropdown (scoped by trigger type if needed).
- **New trigger source:** (1) Add source value to canvas dropdown + validation whitelist. (2) Add branch to `_enrollment_query_for_source()` with correct join topology. (3) Ensure `__source_row_id` is set to the triggering row PK — Condition node uses it for live re-reads.
- **New `CrmEvent` hook:** In the service function, after outer transaction commits, call `await _maybe_start_flow(conn, event_type="stage_changed", customer_id=customer_id, entity_id=lead_id)`. Note: wiring to all planned service functions is not yet complete.

⚠ **Operator-canon maintenance.** Condition node operators are stored as strings (`eq`, `neq`, `contains`, `gt`, `lt`). The engine switches on these exact strings. A renamed operator (e.g. `not_eq`) silently falls through to `false_output` for all existing runs. Any change must be coordinated across the canvas dropdown, validation schema, and engine handler simultaneously.

7. Roadmap + API Reference

Roadmap

Phase / Slice	Unlocks	Key work items	Status
Phase 2 — Two-way chat	Staff see + reply to client WhatsApp messages inside CRM	Webhook receiver; <code>wa_conversations</code> + <code>wa_messages</code> tables; CRM chat panel; JID → customer lookup; auto-update <code>crm_leads.follow_up_status</code> on inbound reply	Planned
Slice 3 — Inbound bots	Keyword-triggered conversation bots on canvas	<code>InboundKeyword</code> engine handler; <code>Wait(reply)</code> handler; webhook → flow resume wiring	Deferred (after Phase 2)
Slice 3 — Campaigns	Staff trigger a flow against a filtered audience immediately	Trigger-now endpoint; audience filter builder; rate-paced batch enrollment	Deferred (after Phase 2)
Slice 2 additions	Canvas usability + activity log	Activity log (last 50 <code>wa_outbox</code> rows/customer); <code>UpdateCRMField</code> handler; <code>CrnEvent</code> hook wiring; 90-day retention cleanup; consent capture UI	In design

API endpoint reference

All endpoints under `/api/v1/whatsapp/`. All require `Authorization: Bearer <JWT>`.

Method	Path	Purpose
POST	<code>/send</code>	Staff-initiated manual send. Full guardrail chain. Returns <code>{ "evolution_msg_id" }</code> or error per guardrail table in §3.
GET	<code>/instance/status</code>	Connection state of active Evolution API instance + Redis daily send counter.
GET	<code>/flows</code>	List all <code>wa_flows</code> rows (id, name, trigger_type, status, is_active, version, created_at). Newest-first.
POST	<code>/flows</code>	Create new flow; <code>status='draft'</code> , empty <code>draft_data</code> .
GET	<code>/flows/{id}</code>	Fetch single flow including <code>draft_data</code> and <code>live_data</code> .
PUT	<code>/flows/{id}/draft</code>	Auto-save. Accepts <code>reactFlowInstance.toObject()</code> payload; writes to <code>draft_data</code> . Called debounced every 2 s.
POST	<code>/flows/{id}/validate</code>	6-check validation on <code>draft_data</code> . Returns <code>{ "valid": bool, "issues": [{"node_id", "message"}] }</code> .
POST	<code>/flows/{id}/publish</code>	Validates + copies <code>draft_data</code> → <code>live_data</code> ; increments <code>version</code> ; <code>status='published'</code> .
PATCH	<code>/flows/{id}</code>	Patch scalar fields: <code>name</code> , <code>is_active</code> , <code>status</code> (archiving). Does not accept data fields directly.
POST	<code>/flows/{id}/simulate</code>	Accepts <code>{ "customer_id", "context_overrides", "simulated_replies" }</code> . Rolled-back transaction. Returns <code>{ "trace", "would_send" }</code> .

G. Glossary

Baileys	Open-source TypeScript library that reverse-engineers the WhatsApp Web WebSocket protocol. Evolution API uses Baileys as its transport. Operates outside WhatsApp ToS — see <code>07-risks-compliance.md</code> .
instance	An Evolution API "instance" = one WhatsApp session (one phone number). SolvetaxAdmin v1 requires exactly one active instance named <code>primary</code> , configured in <code>wa_instance_config</code> .
guardrail	One of the three checks in <code>send_service.send()</code> : consent, quiet hours, daily cap. All three must pass before any message is dispatched to Evolution API.
outbox	<code>wa_outbox</code> table. Decouples message creation (SendMessage handler) from HTTP delivery (outbox dispatcher step 14c). Every outbound send writes an outbox row first.
DryRunSink	A <code>MessageSink</code> protocol implementation that records sends in-memory and returns a fake <code>evolution_msg_id</code> . Used by the simulation endpoint and all unit tests. Contrast with <code>EvolutionSink</code> (real HTTP).
FAIL-CLOSED	When Redis is unavailable the daily-cap check blocks all sends rather than allowing uncounted sends through. The one deliberate exception to the codebase's fail-open Redis policy.
claim-then-release	Outbox dispatch pattern: a short transaction sets rows to <code>status='sending'</code> and commits immediately (locks held milliseconds), then HTTP calls run lock-free. Prevents long-held locks blocking the scheduler.